

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امرالہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 26/30

Session 26. Member-wise/Uniform initialization, In-class member initialization, and Introduction to Destructors

- Class members initialization vs. assignment
- C++11 In-class member initialization
- Special member functions: Destructors
- References and further readings
- Q&A

120 min (incl. Q & A)



The **D**ate Class so far ...

```
class Date {  
public:  
    Date(int = 0, int = 0, int = 0); // day, month, year  
    Date(const char*); // Date in string rep.  
    int day() const { return d_; }  
    int month() const { return m_; }  
    int year() const { return y_; }  
    void add_year(int y) { y_ += y; }  
    void add_month(int m) { /* ... */ }  
    void add_day(int d) { /* ... */ }  
private:  
    int d_, m_, y_;  
};
```

The **V**ector Class so far ...

```
class Vector {  
    int size_;  
    int* elem_; // pointer to the 1st element (of type int)  
public:  
    Vector(int s); // constructor: allocate s ints,  
                    // let elem point to them  
                    // store s in sz  
    int size() const { return size_; }  
    int get(int i) const { return elem_[i]; }  
    void set(int i, int val) { elem_[i] = val; }  
};
```

Recall

class member Initialization

- Initialization vs. Assignment
- In C++, an initialization occurs when a new object is created; an assignment changes the value of an existing object (no new object is created).

```
Thing t = x;    // Initialization (new Thing created)
t = x;         // Assignment (value of existing Thing changed)
```

```
class Date {
public:
    Date(int d, int m, int y)
    {
        d_ = d; m_ = m; y_ = y; // assignment
    }
    // ...
private:
    int d_, m_, y_;
};
```

Assignment

vs.

```
class Date {
public:
    Date(int d, int m, int y) :
        d_(d), m_(m), y_(y) // initialization
    {
    }
    // ...
private:
    int d_, m_, y_;
};
```

Initialization

- Order of initialization
- Uniform initialization:

Uniform initialization

C++98

C++11

vs.

```
int i = 1; // assignment style
int j = i; // initialization
int* ip = &i;
string a[] = { "foo", "bar" };
vector<int> v{1, 2, 3}; // error
Point p(1, 2); // functional initialization
Point* pp = new Point{1, 2};
```

```
int i{1};
int j{i};
int* ip{&i};
string a[] = { "foo", "bar" };
vector<int> v{1, 2, 3}; // OK
Point p = {1, 2}; // = is optional
Point p2{1, 2};
Point* pp{new Point{1,2}};
```

- The class Date

```
class Date {
    Date(int d, int m, int y) :
        d_{d}, m_{m}, y_{y} // initialization
    {
    }
    // ...
private:
    int d_, m_, y_;
};
```

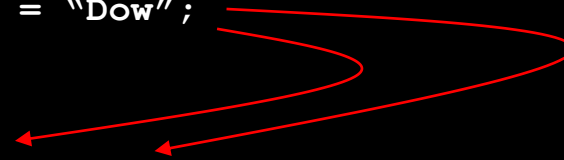
n-class member initializers

- C++98 → Only *static const members of integral types* can be initialized in-class, and the initializer has to be a constant expression.

```
class Values {  
    static const long c = 3e8; // the light speed  
    static const float g = 9.81; // error: not integral type  
    const int i = 10; // error: not static  
    static int j = 11; // error: not const  
};
```

- C++11 → You can initialize non-static data members. Often, all constructors use a common initializer for a member:
- Example:

```
class FinInst { // financial instrument  
    string market = "Dow";  
    string symbol;  
public:  
    FinInst() {}  
    FinInst(string symb) : symbol(symb) {}  
};  
FinInst Microsoft("MSFT"), Oracle("ORCL");
```



In-class member initializers- details

- An in-class member initializer is effectively *syntactic sugar* for an ordinary mem-initializer.
- If a member is initialized by both an in-class initializer and a constructor, only the constructor's initialization is done (it "overrides" the default).
- Another example:

```
class A {  
    int a = 0;  
};
```

It's equivalent to

```
class A {  
    int a;  
public:  
    A() : a(0) {}  
};
```

```
struct Univ {  
    string name;  
    int rank;  
    string city = "unknown";  
};
```

Destructor cont.

A destructor is used to *cleanup* before deleting object.

Destructor

A destructor is often used to *release* resources.

memory, file, lock,

...

- The destructor's name is its classes' name prefixed by '~'.
- A destructor is implicitly called whenever an object goes out of scope or is deleted.
- Destructors do not return value and don't have arguments.
- Destructors can't be overloaded.
- If a user has declared a destructor, that one will be used.
- `Date` and `Point` don't need to define destructor.

R eferences and further readings

[Str13] Bjarne Stroustrup. The C++ Programming Language. 4th edition. Addison Wesley. Reading Mass. USA. May 2013.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

